

Lab3 Report

5. When we changed the goal precision from $\epsilon = 1.0$ to $\epsilon = 0.2$, the planner became harder to finish because the goal region is much smaller. If we only use the normal random sampler, sometimes the RRT takes a long time or cannot reach the goal.

After adding the goal-biased sampling, the planner works much better. The tree can explore globally first, and then it can connect to the small goal region more easily. In our tests, the planning time becomes shorter and the success rate becomes higher.

The final trajectory is also more accurate near the goal and looks smoother after using the smoothing function.

6. If we call the "*get_random_sample_near_goal*" with probability 1.0, the RRT algorithm becomes concerning because RRT will lose its random exploration capability. All samples are picked near the goal configuration. This violates the fundamental property of RRT - exploring configuration space randomly, which could significantly increase runtime or make finding a feasible path impossible.

If there exists a collision configuration region between the start and the goal, then using a 1.0 probability of sampling near the goal makes it essentially impossible for the RRT to find a feasible path. Since all samples lie within a small neighborhood around the goal, every extension from these samples eventually intersects the collision region. As a result, the planner loses the ability to bypass the obstacle by exploring alternative parts of the configuration space, which results in a guaranteed failure for finding a feasible path.

Resource consulted:

Python and NumPy documentation for implementation

Team contribution:

Every person contributed equally at 20%.